

TP1 - EJERCICIO PARA EVALUAR**Muestra de funcionamiento lunes 13/4 (asistencia obligatoria)**

El problema tiene como objetivo desarrollar el firmware para un sistema de visualización que gestiona tres tipos de salidas independientes (paralelo y serie) y dos entradas de usuario. El sistema debe demostrar un comportamiento **multitarea cooperativa**, donde ninguna tarea bloquee la ejecución de las demás. La arquitectura del software se basa en la ejecución de un super loop no bloqueante que permita la ejecución de todas las tareas involucradas. En este ejercicio no está permitido el uso de interrupciones ni de periféricos de temporización.

multitarea cooperativa: los led van a responder a los pulsadores, que manejan al cambio de la secuencia. Es decir, el loop tiene en cuenta las entradas (pulsadores) y las salidas comportamiento de leds (en todo momento)

Diseño del esquema electrónico en Proteus para simulación:

1. Conectar 8 diodos LED al puerto D del MCU (utilizar los componentes “*active animated led model*” en modo *analog*).
2. Calcular la resistencia serie para limitar la corriente a 10mA en cada uno teniendo en cuenta la caída de tensión V_{LED} (*forward voltage*) del color elegido, por ejemplo: rojo=1.8V, verde=2.2V amarillo=2.0V, azul=3.0V, etc (editar las propiedades del modelo de componente acorde al problema)
3. Conectar dos pulsadores (*button*) a los terminales PORTC0 y PORTC1 del MCU.
4. Utilizar el componente Neopixel LED (modelo WS2812x8) conectado a la salida PORTB0 (8 Leds Neopixel*).

Diseño de software bare metal con AVR-GCC:

1. Realice el programa para que el MCU encienda los 8 LEDs del puerto D en dos secuencias de encendido distintas:
a-Enciende un LED a la vez y lo hace desde el LSB hacia el MSB repetitivamente.
b-Enciende un LED a la vez empezando desde el MSB y rebotando en los extremos repetitivamente
El retardo de visualización debe ser de 100ms en ambos casos.
generar una ffunction delay ns (porque la libreria solo tiene ms)
2. El programa comienza con la secuencia a) hasta que el usuario quiera cambiarla presionando el pulsador en PORTC0. En este caso pasará a ejecutar la secuencia b). Si presiona el pulsador nuevamente vuelve a la primera secuencia y así sucesivamente.
3. Simultáneamente, el código debe controlar el encendido de los 8 LEDs Neopixel conectados al PORTB0 en las secuencias:
c-Enciende LEDs pares en rojo y luego LEDs impares en color azul repetidamente.
d-Enciende en color verde un LED a la vez de derecha a izquierda repetitivamente
El retardo de visualización debe ser de 150ms en ambos casos. *refrescar cada 150 ms*
4. Si el usuario presiona el pulsador en PORTC1, el programa deberá intercambiar las secuencias c y d que se encuentra en ejecución.

Evaluación:

- **Demostración presencial:** El grupo deberá mostrar la simulación del problema propuesto y explicar claramente cuáles son los razonamientos y herramientas que aplicaron para resolver las distintas partes del programa. Deberán describir el conexionado, el funcionamiento de los periféricos utilizados, la descomposición en funciones del programa, las bibliotecas utilizadas, cuáles son las tareas que lleva a cabo el MCU y la temporización que requieren las mismas.
- **Entrega de Código fuente y Simulación:** un integrante de cada grupo deberá subir al moodle un archivo con formato “TPX_GrupoYY.zip” conteniendo el archivo de simulación Proteus (versión 8.12), y el proyecto completo desarrollado en Microchip Studio con los archivos fuentes para proceder con la calificación.
entregar el vrj de proteus version 8.12 (si o si)

* <https://cdn-shop.adafruit.com/datasheets/WS2812B.pdf>